Urban Mobility Data Explorer

Technical Report

Team: Gary · Espoir · David · Merci

Stack: Python · Flask · MySQL · Vanilla JS · Chart.js · Leaflet.js

1. Problem Framing and Dataset Analysis

It is the NYC TLC Yellow Taxi trip data for January 2019, which consists of 7 million trips (18 columns) including fares, distance, number of passengers, zone IDs and timestamps. Each LocationID is associated in a zone lookup table and shapefile map to a borough and a zone name, and a geographic polygon.

Data Challenges Identified

- The above densities indicate that 63% of rows (4.8M) are not error (structural) and are blank, but were filled with 0.0.
- Trips with a negative duration or distance, or a speed > 100 mph or the number of passengers < 1 or > 6
- Any dates before or after January 2019 (even 2088 and 2001)
- The data was done by dissolving the two zones (IDs 56, 103) that are stored as multi-polygon.

Cleaning Assumptions

No records were taken other than when there was a rule that was clearly and justly stated. There was no limit on distances over 100 miles, indeed it was dropped otherwise it would be “invented” data. A zero count of passengers was considered as an invalid value. 182,195 rows removed (2.38%), leaving 7,485,597 clean trips.

Rule	Rows Removed
Non-positive duration	6,294
Non-positive distance	48,801
Negative fare	5,698
Implausible passenger count	115,626
Implausible speed	5,236
Outside January 2019	508
Total	**182,195**

Derived Features

Trip Duration (in Minutes) Average Trip Speed (in mph) (median: 10.12 mph) Fare per Mile Tip per Percent (for cards only; cash tips not collected by TLC) Is_cross_borough Pickup Hour Pickup Day of Week.

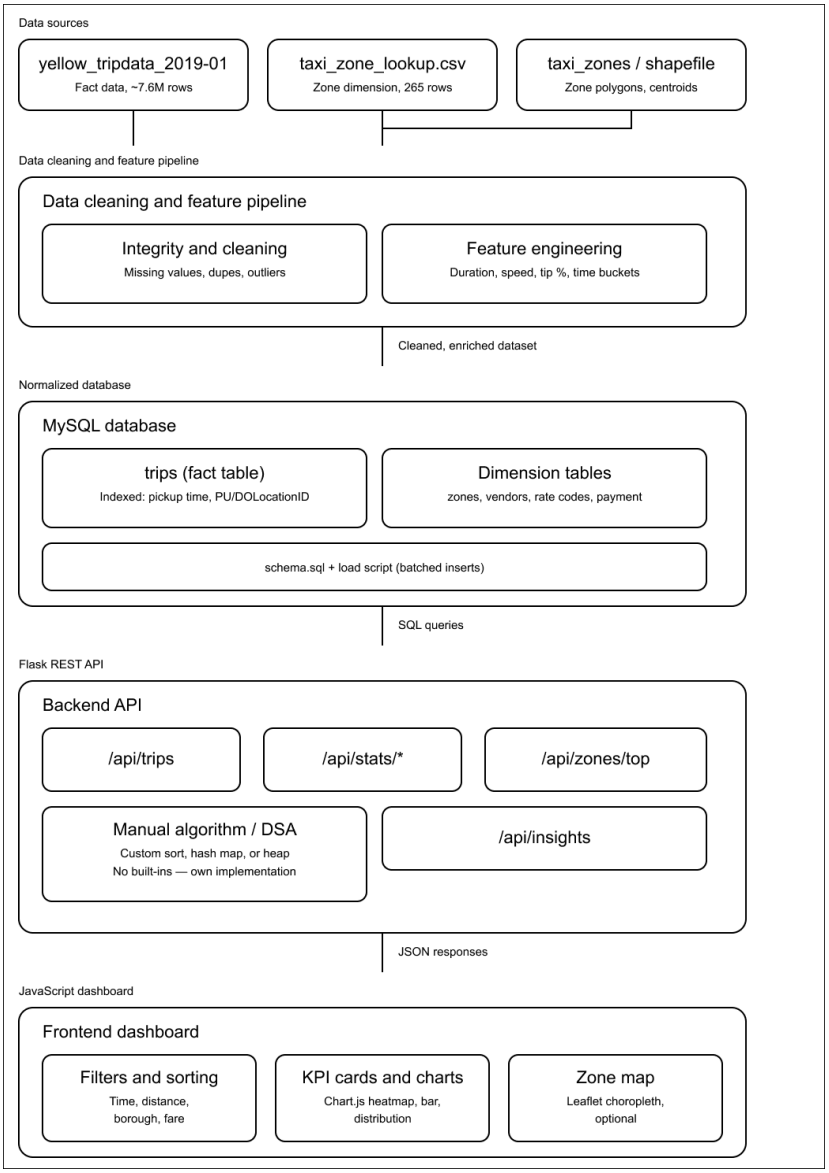
Unexpected Observation

Trip Duration (in Minutes) Average Trip Speed (in mph) (median: 10.12 mph) Fare per Mile Tip per Percent (for cards only; cash tips not collected by TLC) Is_cross_borough Pickup Hour Pickup Day of Week.

2. System Architecture and Design Decisions

Data Flow

Raw CSV + shapefile → Python cleaning pipeline (pandas/geopandas) → MySQL → Flask REST API → JavaScript dashboard (Chart.js + Leaflet)



Stack Choices

- **Python** for the pipeline and API : one language for both layers, native shapefile support via geopandas
- **MySQL** over SQLite : 7.5M rows benefit from connection pooling and enforced foreign keys on Render/PlanetScale

- **Flask** : lightweight, explicit, organized into four Blueprints (trips, stats, zones, insights)
- **Vanilla JS** : no build step, no bundler; deploys as a plain static folder

Schema (3NF)

One fact table, connected to four dimension tables, via foreign keys. These integer codes in the raw data are decodable in dimension tables. Normalized zones means that the borough string isn't repeated across 7.5M rows.

Indexes for trips: pickup_datetime, pu_location_id, do_location_id, payment_type_id, each for a specific actual query path that is used by the API. All filtered requests would be full table scans if they weren't there.

Trade-offs

- SQLite vs MySQL: SQLite was easier to install on the local machine and MySQL was chosen for production concurrency and hosted machine. The cost is required to be set in the .env file and can be found in the README.
- Pre-computed -- at run time, this is very fast, but at startup the loading is expensive (e.g. trip_duration_min, is_cross_borough, etc.).

3. Algorithmic Logic and Data Structures

Problem Statement

Rank all pickup/dropoff zones by trip count and return the top N (max 50) without using built-in sorting, Counter, or heapq.

Approach

Two hand-built structures in backend/algorithms/topk.py:

- **HashMap (separate chaining, polynomial rolling hash)** — counts trips per zone in one O(n) pass
- **MinHeap (fixed capacity k)** — maintains the top-k counts as zones are fed in; evicts the minimum when a larger count arrives

Final ordering uses a hand-written insertion sort over $k \leq 50$ items.

Pseudo-code

```
for each (zone_id, weight) in trip_rows:   // O(n)
   map.increment(zone_id, weight)

for each (zone_id, total) in map.items():  // O(m log k), m ≤ 265 zones
   heap.offer(total, zone_id)

return heap.sorted_desc()                  // O(k²), k ≤ 50 — negligible
```

Complexity

Time: $O(n + m \log k)$ **Space:** $O(m + k)$ — better than a full sort $O(m \log m)$ whenever $k << m$, which is always true here.

4. Insights and Interpretation

The busiest time of day for brands is Insight 2 — 6 PM.The peak of brand demand is between Insight 3 and Insight 6 (3 PM – 6 PM). This query will return the most popular hour or hour range, the next most popular hour or hour range and so on. Each month there are one peak hour.

The demand heatmap shows a regular double peak over the weekdays (8-9 AM and 5-8 PM) with a more level pattern during the weekends, though at an earlier time of day than the weekdays. The demand for yellow cab is highly related to the normal working day.

The evening peak is more distinct than the morning peak; in other words, the time of employees arriving to work in the morning is varied, but the time they leave work is the same. No fleets will be allowed in Midtown and Lower Manhattan between 17:00-20:00. 89% of Trips stay within one Borough (Insight 2) To get the percentage of the number of rows where is_cross_borough = 1 use SUM(CASE WHEN is_cross_borough = 1 THEN 1 ELSE 0 END) / COUNT(*). Only 10.71% of trips cross a borough boundary (801,721 trips). This choropleth map shows Manhattan's largest value for pickups.

The boroughs in NYC do not affect the functioning of the yellow taxi service, which is mainly intended for local trips within the boroughs. There are more cross-borough trips than average in the airports (JFK/Newark) codes. Other modes of transport are used to connect to other parts of the outer boroughs.

When paying for a motorized service with cards, users tip about 20% and cash tips are hidden. I've been finding that the issue is caused by the AVG function.I've noticed that the problem is due to the AVG function. The average tip amount on cards is 18-22%. Cash trips exhibit 0% as that is simply due to the fact that the TLC system does not keep track of cash tips. For the card user there are pre-filled tips on the in-cab terminal (15/20/25%). The concentration of the card tips at 20% suggests that the middle card tip, a measurable variable to consumers, influences the consumer behavior at a mass scale e.g., millions of transactions.

5. Reflection and Future Work

Technical Challenges

To ensure iterations remained fast, explicit dtypes and Parquet intermediates were used for large file processing with 657 MB. Between MySQL vs SQLite, I would have preferred SQLite locally, with only MySQL being used during deployment, but that would have made it harder for all the members of the team to set up their credentials. Shapefile conversion: One-off shapefile-to-GeoJSON conversion (should have been automated in the pipeline).

Team Challenges

When the interfaces are defined in advance, such as schema columns, JSON response shapes, the work split was successful. The biggest issue was frontend–backend the names of the filter parameters varied between the two, and the zones endpoint had no lat/lon coordinates at first. This is because if there was an explicit API contract table in Sprint 0, most of the rework would not have been necessary.

Future Improvements

- This retrieves 7.5M rows on each request, as it currently does.
- Move data from disk to memory for larger data sets.

- Move data from disk to memory for larger data sets. Identifying supply gaps by time of day by clustering the zone level demand.

- To prevent broader public release of the API, authentication and rate limiting will also be implemented.